LIBRARY MANAGEMENT SYSTEM

UML Diagrams

Documentation for the C++ console-based Library Management System. This PDF presents the system's use cases, class structure, issue-book sequence, return-book sequence, and overall activity flow.

Page	Diagram	What it shows
2	Use Case Diagram	Librarian/member interactions with the system
3	Class Diagram	Book, Node, BookList, Member, and Transaction
4	Issue Book Sequence	Validation and book issue process
5	Return Book Sequence	Return process and overdue fine calculation
6	Activity Diagram	High-level application workflow

1. Use Case Diagram

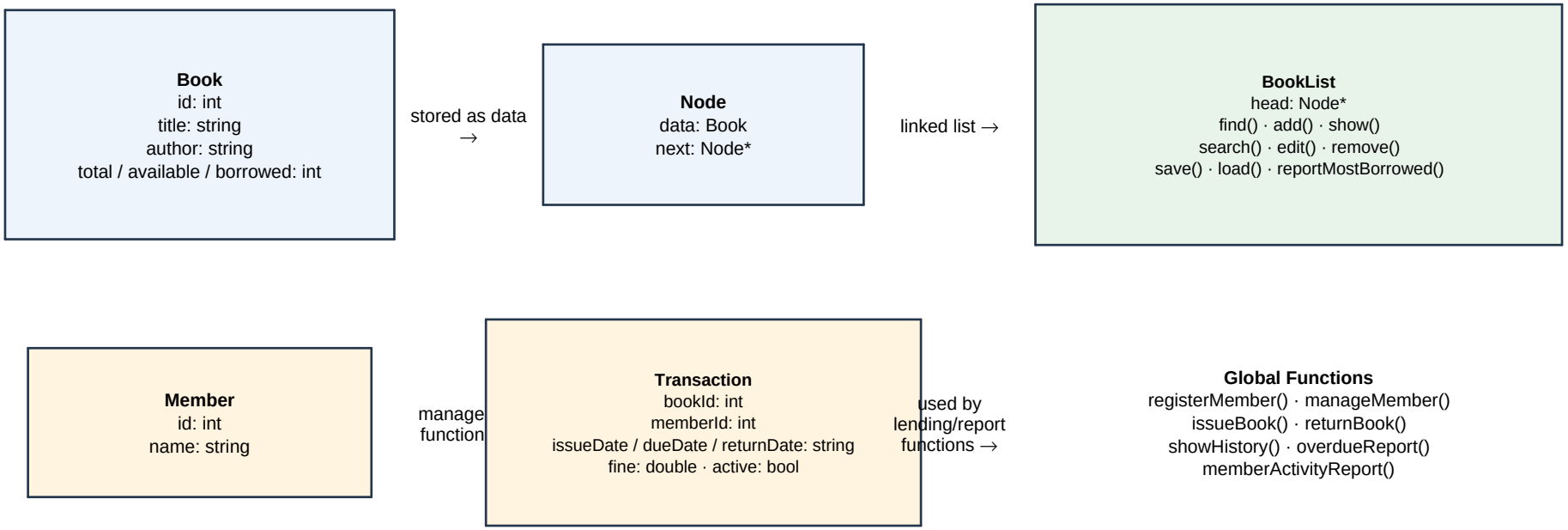
Actors: Librarian and Member

Librarian	Library Management System	Member
	Manage Books (Add / Edit / Delete)	
	Search and View Books	Search and View Books
	Manage Members (Register / Edit / Delete)	
	Issue Book	
	Return Book and Calculate Fine	
	View Borrowing History	View Borrowing History
	Generate Reports (Overdue / Most Borrowed / Member Activity)	
	Save Records to Text Files	

The librarian handles book/member administration, lending, reports, and saving. Members can search/view books and view their borrowing history.

2. Class Diagram

Core structures and relationships reflected in the supplied C++ source.



Implementation notes: books are stored in a custom singly linked list. Members and transactions are held in vectors while the program runs. The configured borrow limit is 3 active loans, and the fine is Rs. 5 per overdue day.

3. Sequence Diagram — Issue Book

Librarian	Librarian Menu	issueBook()	BookList	Member Registry	Transactions
Select Issue Book	Open issueBook()				
		Request book ID and member ID			
Enter IDs			find(bookId)		
			Return book / not found	findMember(memberId)	
				Return member / not found	countActive(memberId)
		Check book/member, availability, and 3-loan limit			Return active loan count
Enter issue and due dates		Validate date order			
		If valid: create active transaction	available--, borrowed++		Add transaction
Receive result		Show success or validation error			

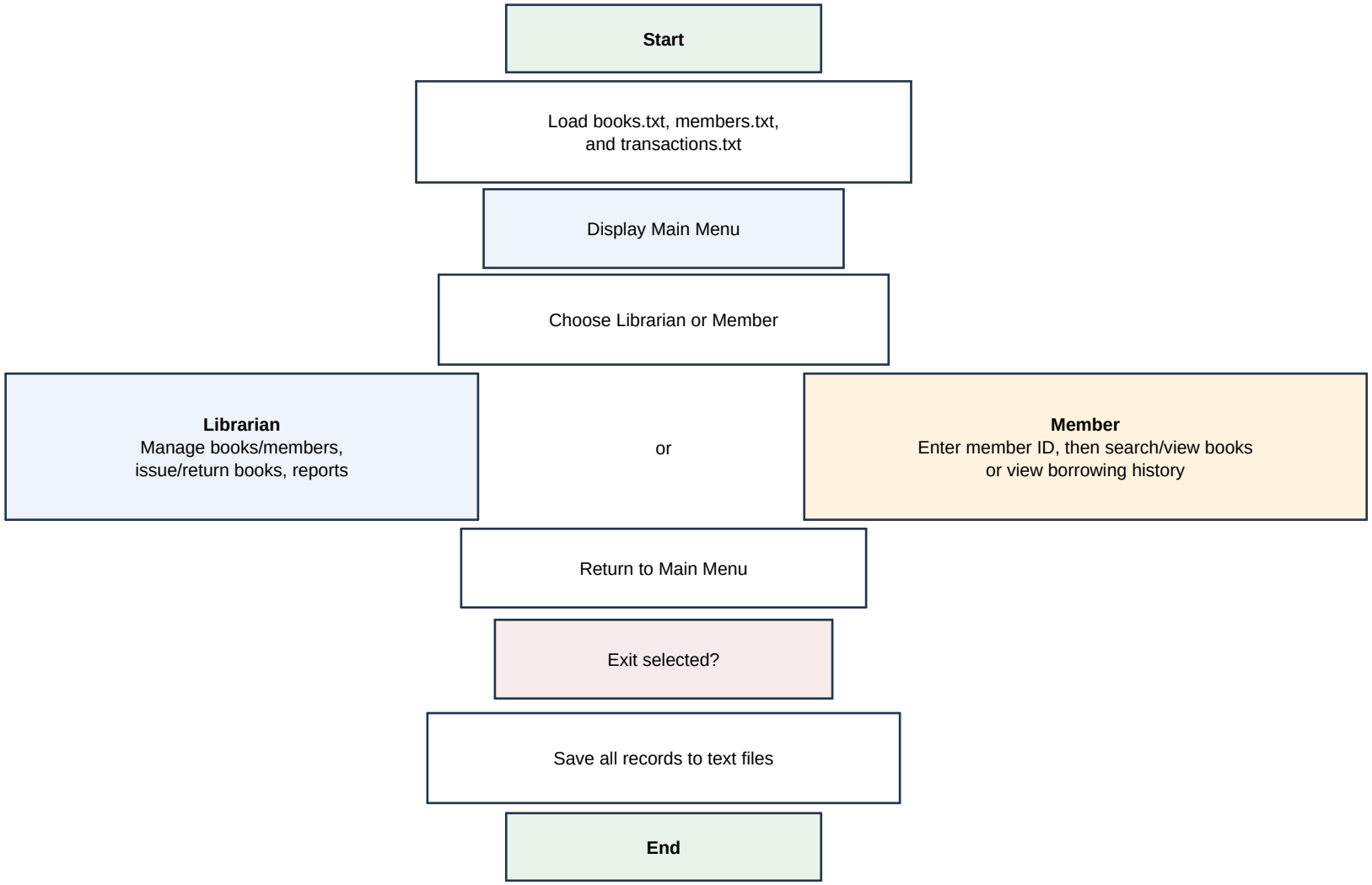
Validation checks include valid book/member records, available copies, a maximum of 3 active loans, and a due date that is not earlier than the issue date.

4. Sequence Diagram — Return Book

Librarian	Librarian Menu	returnBook()	Transactions	BookList
Select Return Book	Open returnBook()			
Enter book ID and member ID		Search matching active transaction	Find active record	
		If no record, show error	Return not found	
Enter return date		Calculate overdue days and fine		
		Fine = overdue days × Rs. 5	Update fine, return date, active=false	
		Find book record		find(bookId)
		If book exists, increase available copies		available++
Receive confirmation and fine		Display return result		

A transaction is marked inactive after return. The fine is based on overdue days at Rs. 5 per day.

5. Activity Diagram — Overall Application Flow



The application loads saved data at startup and writes records back to text files when saving or exiting.